PLAYROOM

Master Architecture & Delivery Roadmap

The neutral room where humans and AI agents — any provider's — work together under enforced permissions, with receipts.

Owner: Prince · Version: 1.0 · Date: 24 July 2026 · Status: build-ready

Executive Position. v1.0 turns the Playroom blueprint (v0.1 → v0.2) and the post-feedback pitch deck into a build-ready delivery plan. It follows the structure that worked for the Jarvis v3 roadmap — concrete schemas, staged pipelines, explicit budgets, failure modes, binary exit criteria — and goes one level deeper: every phase is decomposed into vertical slices of two to four focused days, each with a user-visible exit and a named guard against the premortem it is most likely to trip. Three standing laws inherited from the premortems govern everything below: every milestone is executable with assets Prince fully owns (PM9); every slice ends with something demonstrable (PM5); and the calendar bends around final-year exams instead of pretending they do not exist (PM8).

Audit Acceptance & Changes from Blueprint v0.2

Two audits shaped this document. The blueprint's own premortems (eight scenarios, later nine) forced pre-commitments that appear below as guards. A YC-partner-style critique of the pitch deck forced product changes that appear below as slices. The key deltas from v0.2:

- **Wedge is code review on our own repository.** PryceBridge is deferred; every exit criterion is executable with owned assets (new PM9 rule).
- **Name is Playroom;** Jerry is a contributor (reputation and screening ML), not a co-founder. The plan assumes solo velocity and treats his lanes as async and off the critical path.
- **Two product features promoted to first-class slices:** export-a-chat-into-a-room (S1.7) and ambient mode via MCP connector (Phase 4, gated).
- **Commercial commitment added:** £99/team/month pilot, three paying teams by end of November (S2.9). The roadmap is now accountable to a revenue date, not only a demo date.
- **Latency and performance budgets defined per operation** (Section 7) — without targets you cannot tell broken from slow.
- **Telemetry and audit schemas written as DDL** (Section 17), not prose.
- **Failure modes specified per component** (Section 12.2), including the fail-closed rule for the permission engine.
- **Permission documents and prompts are versioned as code** with hash-per-call reproducibility (Section 18.1).
- **A solo-founder reality check replaces the hardware reality check** (Section 20): hours, velocity, burn, and the exam window, honestly.

Reaffirmed Position. The trust fabric as a chokepoint is unchanged and remains the moat: enforcement lives server-side on a plane no model provider controls. Screening reduces incident frequency; permissions bound incident cost. A fully hijacked agent still cannot exceed its permissions

— that sentence is the product.

1. Product Vision

Playroom is a persistent multiplayer room where humans and AI agents — from any provider — are first-class, @-taggable members. It behaves like a governed workspace, not a group chat with bots: agents carry their principal's context and no one else's, act only under signed permissions, can escalate to humans with priced interrupts, and leave signed receipts for anything commitment-shaped.

Capability	Target behaviour	Trust principle
Room threads	Humans and agents share one persistent thread; roster is invite-only	Common ground is shared by membership
Tagging	@name resolves any member; agents address humans identically	Symmetry of address, asymmetry of authority
Summoning	Agents are silent until tagged, task-moved, or scheduled	Silence by default (PM1)
Interrupts	Agents escalate as BLOCKER / DECISION / FYI; recipients can downgrade	Interrupting a human spends budget
Handoffs	“@Sol, take review” transfers task, state, artifacts, permission ref	Maps onto the A2A task lifecycle
Permissions	Signed, deny-by-default, evaluated server-side on every action	A hijacked agent cannot exceed them
Receipts	Commitments render as co-signed cards on a hash-chained log	Provable history both sides keep
Export-a-chat	One tap turns a live Claude/ChatGPT thread into a room born with context	Onboarding is a conversation you already have
GitHub bridge	Rooms mirror PRs; agent reviews land as PR comments	The counterparty installs nothing (PM4)
Ambient mode	Side panel + MCP connector inside existing chat surfaces (Phase 4)	Reduce switching costs, don't add an app

2. Non-Negotiable Architecture Principles

- Enforcement server-side, never model-side: the model is never the security boundary; permission evaluation happens in the fabric on every cross-boundary action.
- No bypass path: there is no route from a room to a provider adapter that does not pass through the fabric. This is a structural property of the code layout, not a convention.
- Context never crosses principals: assembly can reach the common ground and the summoned agent's own principal store only. A foreign store being reachable is a CI-blocking test failure, not a bug ticket.
- Deny by default: an action not explicitly granted by a permission document is blocked. Unknown action types are blocked. Engine unavailable means cross-boundary actions are blocked (fail closed), reads may continue.

- Silence by default: agents never speak unprompted. Any feature pitch containing “agents could chat about…” is rejected on sight (PM1).
- Receipts for anything commitment-shaped: merges, acceptances, approvals, spends. Never prose.
- Provider-agnostic core: only adapters know provider names. The room, fabric, and data model never do.
- Demo-first delivery: every slice ends in something a camera can see. A slice with no visible outcome is two slices badly cut.
- Assets-owned rule: every phase exit and the canonical demo run on the founder's own repo, agents, and accounts (PM9).
- Boring infrastructure: the novelty budget is spent on the fabric and the room mechanics; everything else is the most boring available option (PM5).
- Spend is visible: per-room budgets and per-summon cost render in-thread. Cost transparency doubles as babble suppression (PM1, PM7).
- Append-only audit: history is hash-chained; the daily root is emailed to principals. Silent rewriting must be detectable by anyone with an inbox.

3. End-State System Architecture

Flow: member input or bridge event → room service → trust fabric (stamp → screen → permit → egress) → agent gateway → provider adapter or bridge → streamed back through the fabric → room → receipts, audit, telemetry.

Layer	Components	Purpose
Clients	Next.js web app; side-panel surface later (Phase 4)	Where members read, tag, approve, and watch spend
Room service	Threads, roster, presence, tags, tasks, interrupts — WebSocket + Redis pub/sub	Normalises all member-visible events
Trust fabric	Identity stamping · inbound screening · permission engine · egress control · audit chain	The chokepoint: every cross-boundary hop passes through; nothing else may talk to adapters
Agent gateway	Context assembly, AgentTurn normalisation, streaming, postage metering	Makes every provider look identical to the room
Adapters	Anthropic, OpenAI, A2A endpoint (P3)	Provider-specific translation only
Bridges	GitHub (P2, primary), email (P3, deferred commerce)	Full participation for counterparties who install nothing
Data	Postgres 16 (+pgvector, RLS), Redis, S3-compatible object store	System of record; per-principal context isolation at the database layer

4. The Trust Fabric (Concrete)

The blueprint named the fabric; this section specifies it as four independent, individually testable stages that every cross-boundary message traverses in order. Coupling them creates bugs: a message can be perfectly authentic (stage 1) and still malicious (stage 2); perfectly benign (stage 2) and still unauthorised (stage 3); perfectly authorised (stage 3) and still leaking (stage 4).

4.1 Identity Stamping

The gateway stamps every turn with the agent id, principal binding, and permission document hash before anything else sees it. Names are never self-asserted: a message claiming to be “Sol” that did not enter through Jerry's authenticated adapter session simply does not acquire the stamp, and unstamped cross-boundary messages are dropped at the room service. Ed25519, custodial keys in v1 (Section 14 discloses what that means).

4.2 Inbound Screening

Two layers, run in order, each with a recorded verdict:

- **L1 — provenance framing (structural).** All foreign content is wrapped as data with source labels before reaching any model context; agent prompts are built to treat wrapped content as data, never instructions. Necessary, not sufficient — and logged as such.
- **L2 — detection (statistical).** A rule pack (known injection patterns, role-play preambles, encoding tricks) plus a small classifier served by the Python screening service. Verdicts: PASS, FLAG (deliver + alert), HOLD (quarantine for human release), BLOCK. Thresholds start conservative and are tuned against the corpus in Section 18.

4.3 Permission Evaluation

Deny-by-default, server-side, under 30ms at P95. The evaluation order is fixed and boring on purpose:

```
def evaluate(action, agent, permit):
    if permit.expired() or not permit.sig_valid():   return BLOCK
    if action.type not in permit.scope:              return BLOCK   # unknown = denied
    if action.type in permit.protected_actions:      return CO_SIGN # route DECISION to
principal
    if permit.counterparties == 'roster_only' and \
       action.target not in room.roster:             return BLOCK
    if breaches_limits(action, permit.limits):       return CO_SIGN if
within_cosign(action) else BLOCK
    return ALLOW                                     # every outcome is audited with
permit hash
```

Outcomes: ALLOW proceeds; CO_SIGN pauses the action and raises a DECISION interrupt to the owning principal; BLOCK stops it and notifies both the agent's principal and, for protected attempts, the room; HOLD exists only as a screening verdict, never a permission one.

4.4 Egress Control

Outbound cross-boundary messages are scanned against the sending principal's secret tags and seeded canary tokens (format in Section 13); hits block the message and alert the principal. Postage is debited per agent-initiated message and per interrupt, from a per-principal budget — economics as the last, honest layer: it raises attacker cost, it does not stop a funded one.

5. Permission Documents (Concrete Schema)

Permissions are signed JSON documents, stored versioned under permits/ in git exactly like prompts (Section 18.1). The schema is wedge-agnostic: the same shape carries review scopes today and spend caps when the commerce wedge revives. Example — Sol's live document for the dogfood repo:

```
{
  "permit_id": "pmt_7f3a",
```

```
  "principal": "org:playroom/jerry",
  "agent": "agent:sol.gpt",
  "scope": ["pr.review", "pr.comment", "task.accept"],
  "protected_actions": ["pr.merge", "deploy"],
  "co_sign": { "actions": ["pr.merge"], "by": "principal" },
  "limits": { "interrupts_per_day": 6, "postage_per_day": 200 },
  "counterparties": "roster_only",
  "expires": "2026-11-30T00:00:00Z",
  "sig": "ed25519:jerry..."
}
```

Commerce extension (dormant until the wedge revives): limits gains per_txn, aggregate, and co_sign_over amounts in a named currency. Review-only versus merge is the same shape as negotiate-only versus accept — authority, bounded in advance.

6. Adapter & Bridge Roles Without Bias

Adapter / bridge	Best role	Rule
Anthropic adapter	Founder's primary agent; deep review turns	Never hardcoded as default — the roster decides
OpenAI adapter	Second principal's agent (Sol); cross-provider proof	Same AgentTurn interface, zero special cases
A2A endpoint (P3)	External agents joining under their own Agent Cards	Conformance target, not a rewrite — constructs already map
GitHub bridge (P2)	Counterparties who live in PRs and install nothing	Bridged threads count as full threads in every metric (PM4)
Email bridge (P3)	Deferred commerce wedge; agent-less counterparties	Kept warm, not gold-plated (PM5)

Anti-lock-in rule. The room, fabric, and data model never contain a provider name. Adapters implement one AgentTurn interface (streamed text, tool calls, task actions) and are the only files that change when a provider changes. Every model call records: adapter id, task type, tokens, cost estimate, latency, verdicts, and permit hash — so provider choice stays an evidence question, not a preference.

```
# adapters.yaml — the only place providers are named
adapters:
  - id: claude-main
    provider: anthropic
    transport: sdk-stream
    capabilities: [chat, tool_use, streaming]
    context_window: 200000
    enabled: true
  - id: sol
    provider: openai
    transport: sdk-stream
    capabilities: [chat, tool_use, streaming]
```

```
    context_window: 128000
    enabled: true
  - id: github-bridge
    provider: github
    transport: webhook+rest
    capabilities: [pr_events, pr_comment]
    enabled: false   # flips on in S2.6
```

7. Latency & Performance Budgets

Without explicit targets you cannot tell broken from slow — and in a trust product, slow enforcement invites bypass pressure. Telemetry (Section 17) tracks all of these; a week of P95 drift is a bug, not noise.

Operation	P50	P95	Ceiling	Fail mode
Message fan-out to room members	<120ms	<250ms	1s	degrade to SSE
Permission evaluation	<10ms	<30ms	100ms	fail closed
Inbound screen — L1 wrap + rules	<80ms	<250ms	600ms	HOLD
Inbound screen — with classifier	<350ms	<900ms	2s	L1-only mode + stricter co-sign
Summon: context assembly	<250ms	<400ms	1s	trim window, log
First streamed token (cloud)	<900ms	<1.5s	3s	notify + hold task
Receipt sign + append	<25ms	<50ms	200ms	retry once, else co-sign path
Interrupt push to human device	<600ms	<1s	3s	fall back to in-thread card
GitHub webhook → room event	<800ms	<2s	10s	poll reconcile (S2.6)
Audit append	<10ms	<20ms	100ms	block cross-boundary sends

8. Streaming & Realtime Architecture

- All provider calls stream by default; tokens are batched into message deltas every ~150ms and fanned out over WebSocket, with SSE as the degraded path.

- Every event carries a monotonically increasing room sequence id. Delivery is at-least-once; clients dedupe on event id and reconnect with resume-from-last-id. The server replays from Postgres, so a dropped socket never loses a receipt.
- Interrupt semantics: a human reply to a streaming agent flushes that agent's output queue at the next sentence boundary and re-summons with the new context. Agents never talk over a principal.
- Working indicators are events, not polling: task state changes (working, input-required, done) render as chips the moment they commit.
- Ordering rule: fabric verdicts commit to the audit chain before the message fans out. Members never see a message the fabric has not finished judging.

9. State Management & Persistence

State	Store	Rationale
Rooms, members, messages, tasks	Postgres 16	Structured, queryable, survives everything; source of truth for replay
Permission documents	Postgres + permits/ in git	Runtime copy for evaluation; git copy for versioning, diff, rollback
Audit chain	Postgres append-only table	Hash-chained rows; daily root emailed to principals (cheap external anchor)
Receipts	Postgres + rendered artifact in S3	Queryable and human-readable; verification page reads both
Principal context	Postgres schema-per-principal + pgvector, RLS on	Isolation enforced by the database, not by discipline
Hot presence, stream buffers	Redis	Recreatable on restart; loss degrades presence, never messages
Artifacts (diffs, exports)	S3-compatible	Cheap, immutable, content-addressed
Prompts & permit templates	Files under git	Versioned, diff-able, hash logged per call (Section 18.1)
Config	.env + YAML per environment	Boring on purpose

10. Repository Structure

```
playroom/
  apps/
    web/                  # Next.js room UI
    api/                  # Fastify + tRPC — room service + gateway
  services/
    screening/            # FastAPI — L2 classifier, DLP, reputation (Jerry's lane)
  packages/
    fabric/               # identity, permits, screening client, egress, audit
    adapters/             # anthropic/, openai/, a2a/, github/, email/
    shared/               # event types, AgentTurn, zod schemas
  permits/
    templates/            # review-only.json, commerce.json (dormant)
```

```
  prompts/                # versioned, git-tracked, hash-logged
  infra/                  # fly.toml, docker-compose, migrations
  tests/
    fabric/               # permission table tests, hijack simulation
    assembly/             # foreign-store-unreachable (CI-blocking)
    screening/            # injection corpus
  docs/
    decisions/            # ADRs — every open decision from the blueprint lands here
  scripts/                # seed, canary tools, root-anchor mailer
```

11. Build Roadmap — Zero to Open Beta, in Slices

11.1 What counts as a slice

- Two to four focused days of solo work — if it is bigger, it is two slices badly cut.
- Vertical: touches whatever layers it needs to end in something user-visible. A slice that ends in a library is not done.
- Binary exit: the criterion is a test that passes or a clip that exists, never “mostly works”.
- Filmed: every slice closes with a 30-second screen recording. The P0 demo is then an edit job, not a scramble (PM5).
- Guarded: each slice names the premortem it is most likely to trip, and its tripwire is watched while the slice is live.
- Revertable: a slice merges behind a flag where feasible; rollback is a flag flip or a git revert, never surgery.

11.2 Phase P0 — Spike (Aug 2026, ~2 weeks, pre-term)

Slice	Build work	Exit criterion (binary)	Guards
S0.1	Repo, pnpm workspaces, CI, envs, test harness, ADR template	Fresh clone → one command → app + tests green	PM5
S0.2	Room + message model, WebSocket fan-out, resume-from-last-id	Two browsers converse; kill a socket mid-stream, nothing lost	PM5
S0.3	Anthropic adapter, streamed AgentTurn into the room	@claude → streamed reply visible in-thread	PM7 (cost logged)
S0.4	OpenAI adapter behind the same interface	Same prompt routes through either agent via roster config — no app-code change	lock-in rule §6
S0.5	Summon rule v0: tag-only activation, one turn per summon	20-case test: zero unprompted agent messages	PM1
S0.6	Demo cut: script beats 1–4 live, record	The 90-second video exists and is watchable	PM5 · phase exit

11.3 Phase P1 — Room MVP (Sep 2026, ~4 weeks, term begins)

Slice	Build work	Exit criterion (binary)	Guards

S1.1	Principals, roster, invites; agent ↔ principal binding in data model	Sol cannot exist in a room without Jerry's authenticated enrolment	PM2 · §4.1
S1.2	Identity stamping at the gateway; unstamped drops	Spoof test: forged “Sol” message never renders	PM2
S1.3	Tasks + handoff object with A2A-shaped states	“@Sol take review” moves task with state + permit ref, logged	§6 mapping
S1.4	Interrupts: BLOCKER / DECISION / FYI + one-tap downgrade	Downgrade decrements the agent's interrupt budget, visibly	PM1
S1.5	Context scopes v0: per-principal store, assembly with the assert	CI test proves a foreign store is unreachable from assembly — blocking	PM2 · law #3
S1.6	Rolling summary + per-room budget meter in-thread	50-message room summons at <7k tokens; spend visible to all members	PM7
S1.7	Export-a-chat v0: paste Claude/ChatGPT export → room born with history, provenance-tagged as imported	A real prior conversation becomes a joinable room in under a minute	PM4 · deck S5
S1.X	Phase exit: a real PR on the Playroom repo reviewed end-to-end in the room	Clip exists: tag → review → patch → approve	PM5

11.4 Phase P2 — Fabric v1 + Dogfood + First Revenue (Oct–Nov 2026, ~6 weeks)

Slice	Build work	Exit criterion (binary)	Guards
S2.1	Permission engine: schema, signatures, deny-by-default eval	40-case table test passes, incl. hijack sim: injected agent attempts merge → BLOCK; P95 <30ms	PM2 · fail closed
S2.2	Co-sign flow: CO_SIGN → DECISION card → sign → resume	Merge outside Sol's permit pauses until Jerry taps approve	PM6
S2.3	Receipts + hash chain + daily root email to principals	Tamper test: edited row breaks the chain and the morning email proves it	PM2
S2.4	Inbound screening L1 + L2 (rules + classifier via screening svc)	Corpus run: known injections → 0 PASS; benign FPR under target	PM2
S2.5	Egress DLP + canary seeding tools	Planted canary exfil attempt fires block + principal alert	PM2
S2.6	GitHub bridge: webhook ↔ room, review → PR comment, poll reconcile	A maintainer participates from GitHub having installed nothing	PM4
S2.7	Postage budgets + interrupt pricing live	Budget breach degrades to digest mode, never a surprise bill	PM1 · PM7
S2.8	Red-team week: founder attacks own boundary; findings triaged	≥5 findings logged with severity + fix-or-accept decisions; canaries verified end-to-end	PM2 · tripwire
S2.9	Pilot onboarding + Stripe:	Three external teams paying by 30	PM3 · PM4 ·

	£99/team/month, docs, support channel	Nov — or the tripwire fires and P3 re-scopes	deck
S2.X	Phase exit: 20 real PRs through the room; co-sign fired ≥1× in anger	Dogfood dashboard shows 20; audit chain verifies	PM5

11.5 Phase P3 — Open Network Beta (Q1 2027, post-exams)

Slice	Build work	Exit criterion (binary)	Guards
S3.1	A2A conformance endpoint: accept external signed Agent Cards, map task lifecycle	A reference A2A agent completes a task in our room under our permit	§6 · PM3
S3.2	Reputation v0 (Jerry): downgrade counts + postage decay	Chronically mislevelled agent measurably loses interrupt budget	PM1
S3.3	Email bridge for the deferred commerce wedge	A quoted-terms email round-trips into a task without manual parsing	PM4 · kept minimal
S3.4	Orgs, roles, multi-room administration	A pilot team self-serves a second room with scoped permits	PM5 (only if pilots pull)
S3.5	Receipt verification page: counterparty independently checks signatures + chain	Verification works with Playroom's servers treated as untrusted	PM2 · §14 honesty

11.6 Phase P4 — Ambient (gated, not scheduled)

MCP connector exposing rooms, tags, and summons inside Claude and ChatGPT, plus the deep-linked side panel. Gate to open: P2 exit achieved, at least one paying pilot renewed, and a platform-policy review written as an ADR. Ambient is the distribution endgame and the largest platform-risk stack in the plan — it earns a date only after revenue exists (PM3, PM5).

12. Room & Interaction Architecture

12.1 Interrupt Levels and Room Behaviour

Level	Delivery	Behaviour
BLOCKER	Push notification + pinned card	The owning task halts; agent stays silent until answered
DECISION	Queued card + badge	Agent continues on unblocked branches; co-sign requests always arrive here
FYI	Daily digest	Never interrupts; digest also carries spend and downgrade summaries
Downgrade	One tap on any interrupt	Reclassifies, decrements the agent's interrupt budget, feeds reputation (S3.2)

12.2 Failure Modes (per component)

Failure	Detection	Response

Provider outage / 429 mid-task	Adapter error class + retry budget exhausted	Task → held state, persisted; room notified; resume on recovery — task state never lives in provider memory
Permission engine unavailable	Health check or eval timeout >100ms	FAIL CLOSED: cross-boundary actions block, read-only continues; incident banner in room
Screening classifier down	Screening svc health check	L1-only mode flag; co-sign thresholds tighten automatically; logged as degraded
WebSocket drop	Client heartbeat miss	Resume-from-last-event-id replay from Postgres; no gap, no dupes
Redis loss	Connection error	Presence and typing degrade; messages, receipts, audit unaffected
GitHub webhook missed	Sequence gap vs poll	Reconcile poll every 5 min while bridge active; idempotent event ids
Canary token fires	Egress DLP hit	Room freezes cross-boundary sends; both principals alerted; incident runbook opens (disclosure plan from PM2)
Audit append fails	Write error / chain mismatch	Cross-boundary sends block until chain heals — an unaudited action is worse than a delayed one
Interrupt flood	>N interrupts/agent/hour	Rate-limit + auto-downgrade to digest; reputation records the spike
Clock skew	Signed ts vs server ts drift	Server time is authoritative for chain ordering; skew logged

13. Screening Layer Specifics

The blueprint said “injection screening”; this names the parts. L1 wraps every foreign span in a provenance envelope before any model sees it:

```
<foreign source="agent:sol.gpt" principal="org:acme/jerry" verdict="PASS" perm="pmt_7f3a">
  ...counterparty content, delivered as data...
</foreign>
```

- **L2 rule pack:** instruction-override patterns, role-play preambles, encoding and homoglyph tricks, tool-call smuggling. Rules are versioned files; each hit names its rule id in telemetry.
- **L2 classifier:** small model behind the FastAPI screening service, threshold τ tuned on the corpus (Section 18); verdict matrix PASS / FLAG / HOLD / BLOCK with per-room strictness.
- **L3 DLP:** secret-tagged context items matched with normalised fuzzy matching; canaries are generated strings of the form plr_cnry_<base32> seeded into every principal store before any external agent joins (PM2 pre-commitment).
- **Scan policy:** full scan on every cross-boundary hop; sampled scan (1 in N) on intra-principal traffic to keep latency inside Section 7 budgets.

- **Honest limit, in writing:** paraphrase leakage below DLP granularity survives all layers. The mitigation is a small common-ground window and permissions that cap what a leak is worth — not a claim that detection catches semantics.

14. Safety, Privacy & Permission Model

Action class	Default	Examples
Read room / common ground	Allowed	Any member; it is common ground by definition
Post to room	Allowed (postage-metered for agents)	Messages, artifacts, task updates
Task actions in scope	Allowed + audited	pr.review, pr.comment, task.accept
Protected actions	Always co-sign	pr.merge, deploy — and any spend, when commerce revives
Cross-principal context access	Structurally impossible	Not a permission level — assembly cannot express it (S1.5)
External egress (bridge posts)	Permit-scoped + egress-scanned	PR comments, emails
Operator (us) reading rooms	Possible in v1 — disclosed	Custodial keys; no E2E yet; see honesty items below

Approval flow: a CO_SIGN or BLOCKER raises a card with the exact action, the permit line that triggered it, and one-tap approve / deny / downgrade. Unanswered DECISION cards hold their branch; nothing times out into an approval. Emergency stop: /freeze halts all cross-boundary sends in a room instantly, any human member may invoke it.

What v1 honestly is not. Keys are custodial and Playroom is a trusted operator: we can read rooms and the ToS says so — no implied end-to-end encryption. The CA is centralised. Receipts prove events to the two principals and to us; the S3.5 verification page is the first step toward proving them to the world. Hardware-backed keys, principal-held keys, and third-party anchoring are sequenced behind paying demand, in that order (PM5).

15. Memory, Context & Audit System

Context type	Stored in	Purpose
Common ground window	Postgres (messages) + rolling summary	What every summon sees; bounded by budget
Principal store	Schema-per-principal + pgvector, RLS	What only that principal's agents see
Task state + artifacts	Postgres + S3	Survives provider outages and handoffs
Audit chain	Append-only Postgres	Hash-linked truth; daily root anchored by email
Telemetry	Postgres events table (§17)	Cost, latency, verdicts, drift — the self-audit

		substrate

Trigger	Action
On summon	Assemble: frame + window + own store retrieval + task state; assert provenance; log token count
On promote (consent)	Copy item from private store to common ground; write consent event to chain
Window over budget	Fold oldest span into rolling summary; log compression ratio
Nightly	Anchor: email chain root to all principals; run drift queries (§17); assemble FYI digests
Budget breach	Room → digest mode; owner notified; never a surprise invoice (PM7)

16. Cost Engineering

- Per-room daily budget, default £5, visible in-thread to every member from S1.6 onward.
- Rolling summaries from day one; a healthy summon is ~6k tokens in / ~0.8k out — roughly $0.03 at mid-tier pricing; the naive 50k-token replay (~$0.16+) is the PM7 spiral and is designed out, not policed.
- Postage debits every agent-initiated message and interrupt; silence is free.
- Review loops carry iteration caps: an agent may request re-review at most twice per task before a human must touch it.
- Dev token cap £50/month with a hard stop; pilot teams carry per-team caps in their permits.
- Every model call logs estimated cost and prompt hash, so waste is findable and reproducible.

Feature	Cost risk	Mitigation
Agent chatter	Token burn + user cringe	Silence law + postage; spend public in-thread (PM1)
Ageing rooms	Superlinear context replay	Rolling summary + fixed window (PM7)
Review loops	Recursive agent calls	Iteration caps; diff-based context, not full files
GitHub webhook storms	Event floods	Rate limit + reconcile-poll dedupe (S2.6)
Pilot abuse	One team burns the budget	Per-team caps in permits; digest-mode degrade

17. Telemetry & Audit Schemas (DDL)

Two tables. events is the operational log; audit_chain is the tamper-evident record. They are separate on purpose: telemetry is mutable and prunable, the chain is neither.

```
CREATE TABLE events (
  id              BIGSERIAL PRIMARY KEY,
  ts              TIMESTAMPTZ NOT NULL,
  room_id         TEXT NOT NULL,
```

```
  actor_id        TEXT NOT NULL,          -- member (human or agent)
  principal_id    TEXT,
  event_type      TEXT NOT NULL,          -- message|summon|screen|permit|receipt|
interrupt|bridge|error
  direction       TEXT,                   -- inbound|outbound|internal
  screen_verdict  TEXT,                   -- PASS|FLAG|HOLD|BLOCK
  permit_decision TEXT,                   -- ALLOW|CO_SIGN|BLOCK
  urgency         TEXT,                   -- BLOCKER|DECISION|FYI
  adapter_id      TEXT,                   -- from adapters.yaml
  tokens_in       INTEGER,
  tokens_out      INTEGER,
  cost_usd        NUMERIC(10,5),
  latency_ms      INTEGER,
  success         BOOLEAN NOT NULL,
  error_class     TEXT,
  prompt_hash     TEXT,                   -- reproducibility (§18.1)
  permit_hash     TEXT,
  notes           TEXT
);

CREATE TABLE audit_chain (
  seq             BIGSERIAL PRIMARY KEY,
  ts              TIMESTAMPTZ NOT NULL,
  room_id         TEXT NOT NULL,
  actor_id        TEXT NOT NULL,
  event           TEXT NOT NULL,          -- e.g. pr.merge, permit.grant, consent.promote
  body_hash       TEXT NOT NULL,
  prev_hash       TEXT NOT NULL,
  entry_hash      TEXT NOT NULL,          -- H(prev_hash || body_hash || meta)
  sig             TEXT NOT NULL           -- fabric signature
);
```

Weekly drift queries (run nightly, reviewed weekly): P95 latency per operation vs Section 7; permit BLOCK and CO_SIGN rates per agent; screening false-positive rate on FLAGged-then-released items; cost per summon trend; interrupt downgrade rate per agent (feeds S3.2); unprompted-message count, which must remain exactly zero.

18. Testing, Quality Gates & Versioning

Layer	Minimum gate
Permission engine	40-case table incl. hijack simulation, expired permits, unknown actions, roster violations; P95 <30ms in CI
Context assembly	Foreign-store-unreachable test is CI-blocking; provenance assert covered by property tests
Screening	Versioned corpus: known injections → zero PASS; benign set FPR under target; rule ids asserted
Receipts + chain	Round-trip verify; tamper test breaks chain; root-anchor mail renders
Adapters	Error classes, rate limits, missing keys, streaming resume; contract tests against the AgentTurn interface

GitHub bridge	Webhook replay idempotency; reconcile-poll convergence; comment renders on a real PR
Streaming	Kill-socket replay leaves no gap and no dupes; interrupt flush at sentence boundary
Budgets	Breach degrades to digest; postage debits balance; £50 dev cap hard-stops
Latency	Section 7 P95s measured in CI smoke and alerted in prod
Billing (S2.9)	Stripe test-mode E2E; cancel keeps receipts readable

18.1 Prompts and Permits as Code

Prompts and permission templates live in git. Every model call logs the prompt file hash; every permission evaluation logs the permit hash. A behavioural regression is answered with a diff and a revert, not archaeology. Changes ship under feat/prompt, fix/prompt, feat/permit prefixes so the audit trail reads like a changelog.

19. Explicit Non-Goals

What Playroom is NOT, before scope creep does damage:

- Not an always-listening ambient agent platform. Agents are silent until summoned; ambient mode (P4) is a surface, not a behaviour change.
- Not autonomous spend. Money moves only behind a co-sign, and no money moves at all until the commerce wedge revives.
- Not a blockchain. A Postgres hash chain plus an emailed root gives the property needed.
- Not end-to-end encrypted in v1 — disclosed plainly (Section 14), sequenced behind paying demand.
- Not an agent marketplace, a consumer persona app, or a fine-tuning platform.
- Not a Slack replacement for human-only chat; if no agent is in the roster, use Slack.
- Not multi-provider beyond two adapters until a paying customer forces a third.
- Not enterprise-compliant (SSO, SOC2, DPAs) before ten paying teams exist.
- Not a destination-app bet: the web app is the workbench; distribution is export-a-chat and, later, ambient.

20. Solo-Founder Reality Check

The Jarvis roadmap checked hardware honestly; this plan's scarce resource is founder hours. The table assumes final-year ethical hacking at MMU with exams in January, plus life.

Window	Realistic hours/wk	Implication
Aug (pre-term)	30–35	P0's two weeks are the year's best build window — spend them on slices, not setup
Sep–Nov (term)	14–18	One slice per week is the honest pace; S2.9 (pilots) is the crunch and gets first claim on hours
Dec (revision)	6–8	Ship nothing new; support pilots; write the YC application from artefacts that already exist

Jan (exams)	3–5	Protected. The roadmap schedules zero slices here on purpose (PM8)
Feb–Mar	15–20	P3 slices; interview-ready demo maintenance

Assumption	Honest number
Slice size	2–4 focused days; 28 planned slices ≈ 70–90 focused days
Capacity Aug–Nov	≈ 55–65 focused days → P0–P2 fit only if S3 stays untouched and slices stay small
Jerry's contribution	Async, off critical path: screening corpus, classifier tuning, reputation v0. No slice exit depends on his calendar
Cash burn	Infra ≈ £40/mo (Fly LHR + managed Postgres + object store) + ≤£50/mo dev tokens → <£100/mo pre-revenue
Buffer policy	Pre-agreed cuts in order: P3 date slips first, then S3.4; S2.8 (red-team) and S2.1 (fail-closed engine) are never cut

Conclusion. The plan fits a solo final year — barely, and only with the weekly-demo discipline and the slice-size law holding. The tripwire is unchanged from PM8: the weekly clip slipping twice in a row triggers the pre-agreed cuts, starting with P3's date and never with fabric quality. Revenue by 30 November is ambitious but sits on eight slices, each independently small; if S2.9 misses, the YC application ships on dogfood evidence instead, and says so honestly.

21. Instructions for Future Audits

When asking Claude, GPT, or any senior reviewer to audit v1.0+, the audit must answer:

- Does any code path reach an adapter without traversing all four fabric stages? Prove it from the repo layout, not the diagram.
- Can context assembly ever see a foreign principal store — including through summaries, embeddings, or promoted items — and is the CI test actually blocking?
- Is the permission engine fail-closed everywhere, including partial outages and clock skew?
- Which slice is most likely to slip, and does its slip break the £99 pilot promise or only the internal date?
- Is the GitHub bridge idempotent under webhook replay and reconcile races?
- Where can WebSocket replay and Postgres truth diverge, and who wins?
- Are Section 7 budgets realistic on Fly LHR with a single region, and what breaks first under 10 concurrent rooms?
- Which premortem is currently closest to firing, judged by the tripwire telemetry, and is the plan reacting?
- What should be simplified before more code is written?

22. Immediate Next Actions

Priority	Action	Owner

P0	S0.1–S0.2 this week: repo, CI, room fan-out with resume	Prince
P0	S0.3–S0.4: both adapters behind one interface	Prince + Claude
P0	S0.5–S0.6: summon rule, film the 90-second demo	Prince
P1	Draft permits/templates/review-only.json and the hijack test cases	Prince + Claude
P1	Screening corpus v0: collect injection samples for §18	Jerry (async)
P1	Trademark + domain check: playroom.ai and the existing Playroom game SDK	Prince
P2	Pilot shortlist: ten teams who already pay for two AI subscriptions	Prince
P2	Write ADR-001: fail-closed permission engine (decided, documented)	Prince + Claude